



# **School of Future Tech**

## **Case Study Report**

on

**Case Study 190: Carpenter Workshop Custom Order  
Tracking**

by

**Yash Tambade**

150096725173

## Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References
8. Appendix: DBMS Viva Preparation

## 1. Introduction to the Case Study

A carpentry workshop takes custom furniture orders from customers. Every order carries its own material requirements, production stages and delivery deadline. This case study focuses on designing and implementing a Carpenter Workshop Custom Order Tracking Database using PostgreSQL, with its primary syllabus focus on Database Security and Role Management.

The case study demonstrates how a real workshop's data — customers, orders, production stages and material usage — can be modelled as related tables and protected using role-based access control, so that every order can be traced from customer to final delivery while different members of staff are restricted to only the actions their job requires.

## 2. Problem Statement / Case Background (Abstract)

### Background

A carpentry workshop takes custom furniture orders with material requirements and delivery deadlines. Customers, orders, order stages and material usage must be tracked so that every transaction can be traced from customer to final delivery. In addition, not every staff member should be able to modify workshop data — some should only be able to view it, while others need permission to record new orders and production updates.

### Abstract

This case study presents the design and implementation of a Carpenter Workshop Custom Order Tracking Database in PostgreSQL. Four related tables — customers, orders, order\_stages and material\_usage — are created and linked using primary and foreign keys, and populated with sample data. Database-level security is then layered on top using two roles, workshop\_readonly and workshop\_order\_entry, and two login users, workshop\_viewer and workshop\_employee. The read-only role is tested and shown to be blocked from INSERT operations, while the order-entry role is shown to be able to INSERT new records. An attempt is then made to revoke the INSERT privilege, which reveals an important real PostgreSQL behaviour: only the object owner or a superuser can successfully revoke a privilege, not the grantee itself. The implementation is demonstrated with terminal screenshots covering every step, from table creation to GRANT, REVOKE and backup with pg\_dump.

## 3. Case Study Design

The Carpenter Workshop database is designed as a modular relational schema with role-based access control layered on top, structured in the following stages:

### 1. Data Modeling

- Design four related tables: customers, orders, order\_stages and material\_usage.
- customers is the parent of orders; orders is the parent of both order\_stages and material\_usage.

- Primary keys uniquely identify each row; foreign keys enforce referential integrity between parent and child tables.
2. **Sample Data Population**
    - Insert 5 sample customers, and sample orders, stages and material-usage rows linked to those customers.
  3. **Role-Based Access Control (RBAC) Design**
    - Create workshop\_readonly (SELECT only) and workshop\_order\_entry (SELECT + INSERT).
    - Create login users workshop\_viewer and workshop\_employee and assign each to one of the two roles.
  4. **Access Testing**
    - Verify that workshop\_viewer can SELECT but not INSERT.
    - Verify that workshop\_employee can both SELECT and INSERT.
    - Attempt to REVOKE the INSERT privilege and verify whether it actually takes effect.
  5. **Backup**
    - Export the completed database using pg\_dump for safekeeping.

## Database Schema

| Table          | Key Columns                     | Relationship         |
|----------------|---------------------------------|----------------------|
| customers      | customer_id (PK)                | Parent table         |
| orders         | order_id (PK), customer_id (FK) | References customers |
| order_stages   | stage_id (PK), order_id (FK)    | References orders    |
| material_usage | usage_id (PK), order_id (FK)    | References orders    |

## 4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

Key methods and concepts:

- **Relational data modeling**
  - Entity design for customers, orders, order\_stages and material\_usage.
  - Primary keys and foreign keys to enforce referential integrity.
- **Data Definition Language (DDL)**
  - CREATE TABLE statements with column constraints (PRIMARY KEY, NOT NULL, FOREIGN KEY).
- **Data Manipulation Language (DML)**
  - INSERT statements to populate sample data.
  - SELECT and JOIN queries to verify and trace data across tables.

- **Data Control Language (DCL) / Role-Based Access Control**
  - CREATE ROLE and CREATE USER to define accounts.
  - GRANT to assign SELECT / INSERT privileges to roles.
  - REVOKE to withdraw privileges, and the rule that only an object owner or superuser can do so effectively.
- **Testing methodology**
  - Switching sessions with \c to test each role's actual permissions rather than assuming them.

### **Technology Stack Used for the Case**

- Database: PostgreSQL 18.4 (Homebrew).
- Interface: psql command-line client.
- Operating environment: macOS.
- Backup tool: pg\_dump, producing carpenter\_workshop.sql.
- Data storage: PostgreSQL database carpenter\_workshop.

## **5. Problem Statement / Case Study Implementation Details and Snapshots.**

The implementation was carried out directly in psql, in the following stages:

1. **Database and table creation**
  - CREATE DATABASE carpenter\_workshop; followed by CREATE TABLE for all four tables with their primary and foreign keys.
2. **Sample data insertion**
  - INSERT INTO statements for customers, orders, order\_stages and material\_usage.
3. **Verification queries**
  - SELECT \* FROM statements on each table, and a multi-table JOIN proving full traceability from customer to material.
4. **Role, user and privilege setup**
  - CREATE ROLE, CREATE USER, and GRANT statements for workshop\_readonly and workshop\_order\_entry.
5. **Access control testing**
  - Connecting as workshop\_viewer and workshop\_employee in turn and testing SELECT / INSERT behaviour, including the REVOKE attempt.
6. **Backup**
  - pg\_dump used to export the completed database to carpenter\_workshop.sql.

## Table creation

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    city VARCHAR(50) NOT NULL,  
    order_value DECIMAL(10,2) NOT NULL  
);  
  
CREATE TABLE orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT NOT NULL,  
    pieces INT NOT NULL,  
    order_date DATE NOT NULL,  
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
);  
  
CREATE TABLE order_stages (  
    stage_id INT PRIMARY KEY,  
    order_id INT NOT NULL,  
    stage VARCHAR(50) NOT NULL,  
    stage_date DATE NOT NULL,  
    status VARCHAR(30) NOT NULL,  
    FOREIGN KEY (order_id) REFERENCES orders(order_id)  
);  
  
CREATE TABLE material_usage (  
    usage_id INT PRIMARY KEY,  
    order_id INT NOT NULL,  
    material VARCHAR(100) NOT NULL,  
    quantity DECIMAL(10,2) NOT NULL,  
    unit VARCHAR(30) NOT NULL,  
    FOREIGN KEY (order_id) REFERENCES orders(order_id)  
);
```

## Role, user and privilege setup

```
CREATE ROLE workshop_readonly;  
CREATE ROLE workshop_order_entry;  
  
CREATE USER workshop_viewer WITH PASSWORD 'viewer123';  
CREATE USER workshop_employee WITH PASSWORD 'employee123';  
  
GRANT SELECT ON customers, orders, order_stages, material_usage  
TO workshop_readonly;  
  
GRANT SELECT ON customers, orders, order_stages, material_usage  
TO workshop_order_entry;  
  
GRANT INSERT ON orders, order_stages, material_usage  
TO workshop_order_entry;
```

```
GRANT workshop_readonly TO workshop_viewer;  
GRANT workshop_order_entry TO workshop_employee;
```

An important real-world finding: when REVOKE INSERT ON orders, order\_stages, material\_usage FROM workshop\_order\_entry; was later run while connected as workshop\_employee, PostgreSQL returned warnings that no privileges could be revoked, and a later INSERT by workshop\_employee still succeeded. This is because only the table owner (yash\_tambade) or a superuser can successfully revoke a privilege — not the grantee itself. This is demonstrated in the screenshots below and discussed further in Section 6.

## Sample Data

| customer_id | name   | city     | order_value |
|-------------|--------|----------|-------------|
| 1           | Mehta  | Nagpur   | 6675.00     |
| 2           | Kapoor | Guwahati | 65475.00    |
| 3           | Ramesh | Pune     | 65475.00    |
| 4           | Chavan | Pune     | 61800.00    |
| 5           | Oberoi | Guwahati | 5450.00     |

| order_id | customer_id | pieces | order_date |
|----------|-------------|--------|------------|
| 1        | 1           | 19     | 2024-01-08 |
| 2        | 1           | 14     | 2024-01-19 |
| 3        | 2           | 15     | 2024-02-02 |
| 4        | 1           | 10     | 2024-03-10 |
| 5        | 2           | 20     | 2024-03-15 |

## Snapshots

The following terminal screenshots are presented in chronological order, exactly as captured during the practical, and are unmodified and uncropped.

```
Last login: Fri Sep 26 13:22:22 on ttys000  
yash_tambade@Yashs-MacBook-Air-2 ~ % \psql postgres  
psql (18.4 (Homebrew))  
Type "help" for help.  
  
postgres=# CREATE DATABASE carpenter_workshop;  
CREATE DATABASE  
postgres=# \c carpenter_workshop  
You are now connected to database "carpenter_workshop" as user "yash_tambade".  
carpenter_workshop=#
```

### Screenshot 1: Logging into psql and creating the database

*Shows yash\_tambade logging into psql, running CREATE DATABASE carpenter\_workshop;, and connecting to it with \c.*

```
carpenter_workshop=# CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    city VARCHAR(50) NOT NULL,
    order_value DECIMAL(10,2) NOT NULL
);
CREATE TABLE
carpenter_workshop=#
```

#### Screenshot 2: Creating the customers table

Shows the `CREATE TABLE customers` command with `customer_id` as `PRIMARY KEY`, completing with the `CREATE TABLE` response.

```
carpenter_workshop=# CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT NOT NULL,
    pieces INT NOT NULL,
    order_date DATE NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
CREATE TABLE
carpenter_workshop=#
```

#### Screenshot 3: Creating the orders table

Shows the `CREATE TABLE orders` command, including the `FOREIGN KEY (customer_id) REFERENCES customers(customer_id)` clause.

```
carpenter_workshop=# CREATE TABLE order_stages (
    stage_id INT PRIMARY KEY,
    order_id INT NOT NULL,
    stage VARCHAR(50) NOT NULL,
    stage_date DATE NOT NULL,
    status VARCHAR(30) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(order_id)
);
CREATE TABLE
carpenter_workshop=#
```

#### Screenshot 4: Creating the order\_stages table

Shows the `CREATE TABLE order_stages` command with a foreign key referencing `orders(order_id)`.

```
CREATE TABLE
carpenter_workshop=# CREATE TABLE material_usage (
    usage_id INT PRIMARY KEY,
    order_id INT NOT NULL,
    material VARCHAR(100) NOT NULL,
    quantity DECIMAL(10,2) NOT NULL,
    unit VARCHAR(30) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(order_id)
);
CREATE TABLE
carpenter_workshop=#
```

#### Screenshot 5: Creating the material\_usage table

Shows the `CREATE TABLE material_usage` command with a foreign key referencing `orders(order_id)`.



```
carpenter_workshop=# INSERT INTO customers VALUES
(1, 'Mehta', 'Nagpur', 6675),
(2, 'Kapoor', 'Guwahati', 65475),
(3, 'Ramesh', 'Pune', 65475),
(4, 'Chavan', 'Pune', 61800),
(5, 'Oberoi', 'Guwahati', 5450);
INSERT 0 5
carpenter_workshop=#
```

#### Screenshot 6: Inserting sample customers

Shows the INSERT INTO customers VALUES command for all five sample customers, completing with INSERT 0 5.

```
carpenter_workshop=# INSERT INTO orders VALUES
(1, 1, 19, '2024-01-08'),
(2, 1, 14, '2024-01-19'),
(3, 2, 15, '2024-02-02');
INSERT 0 3
carpenter_workshop=# |
```

#### Screenshot 7: Inserting sample orders

Shows the INSERT INTO orders VALUES command for the first three orders, completing with INSERT 0 3.

```
carpenter_workshop=# INSERT INTO order_stages VALUES
(1, 1, 'Design Approval', '2024-01-10', 'Completed'),
(2, 2, 'Cutting', '2024-02-14', 'In Progress'),
(3, 3, 'Polishing', '2024-03-05', 'Completed');
INSERT 0 3
carpenter_workshop=# |
```

#### Screenshot 8: Inserting sample order stages

Shows the INSERT INTO order\_stages VALUES command for the three sample stages, completing with INSERT 0 3.

```
carpenter_workshop=# INSERT INTO material_usage VALUES
(1, 1, 'Teak Wood', 12.5, 'Cubic Feet'),
(2, 2, 'Plywood Sheets', 8, 'Sheets'),
(3, 3, 'Varnish', 2.5, 'Litres');
INSERT 0 3
carpenter_workshop=# |
```

#### Screenshot 9: Inserting sample material usage

Shows the INSERT INTO material\_usage VALUES command for the three sample material records, completing with INSERT 0 3.

```
carpenter_workshop=# SELECT * FROM customers;
 customer_id | name   | city   | order_value
-----+-----+-----+-----
          1 | Mehta | Nagpur |      6675.00
          2 | Kapoor | Guwahati |     65475.00
          3 | Ramesh | Pune   |     65475.00
          4 | Chavan | Pune   |     61800.00
          5 | Oberoi | Guwahati |     5450.00
(5 rows)
```

#### Screenshot 10: Verifying the customers table

Shows `SELECT * FROM customers`; returning all 5 inserted rows, confirming the data landed correctly.

```
carpenter_workshop=# SELECT * FROM orders;
 order_id | customer_id | pieces | order_date 
-----+-----+-----+-----
          1 |              |      19 | 2024-01-08 
          2 |              |      14 | 2024-01-19 
          3 |              |      16 | 2024-02-02 
(3 rows)
```

**Screenshot 11: Verifying the orders table**

Shows `SELECT * FROM orders`; returning the first 3 rows before the later practical INSERTs.

```
carpenter_workshop=# SELECT * FROM order_stages;
 stage_id | order_id | stage           | stage_date | status 
-----+-----+-----+-----+-----
          1 |          1 | Design Approval | 2024-01-10 | Completed 
          2 |          2 | Cutting         | 2024-02-14 | In Progress 
          3 |          3 | Polishing       | 2024-03-06 | Completed 
(3 rows)
```

**Screenshot 12: Verifying the order\_stages table**

Shows `SELECT * FROM order_stages`; returning all 3 stage rows.

```
carpenter_workshop=# SELECT * FROM material_usage;
 usage_id | order_id | material       | quantity | unit 
-----+-----+-----+-----+-----
          1 |          1 | Teak Wood      |      12.50 | Cubic Feet 
          2 |          2 | Plywood Sheets |       8.00 | Sheets 
          3 |          3 | Varnish        |       2.50 | Litres 
(3 rows)
```

**Screenshot 13: Verifying the material\_usage table**

Shows `SELECT * FROM material_usage`; returning all 3 material rows.

```
carpenter_workshop=# SELECT
o.order_id,
c.name AS customer,
c.city,
o.pieces,
o.order_date,
s.stage,
s.status,
m.material,
m.quantity,
m.unit
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
LEFT JOIN order_stages s ON o.order_id = s.order_id
LEFT JOIN material_usage m ON o.order_id = m.order_id;
 order_id | customer | city   | pieces | order_date | stage           | status   | material       | quantity | unit 
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
          1 | Mehta    | Nagpur |      19 | 2024-01-08 | Design Approval | Completed | Teak Wood      |      12.50 | Cubic Feet 
          2 | Mehta    | Nagpur |      14 | 2024-01-19 | Cutting         | In Progress | Plywood Sheets |       8.00 | Sheets 
          3 | Kapoor   | Guwahati |      16 | 2024-02-02 | Polishing       | Completed | Varnish        |       2.50 | Litres 
(3 rows)
```

**Screenshot 14: Traceability JOIN across all four tables**

Shows a query joining orders, customers, order\_stages and material\_usage, proving every order can be traced from customer to stage to material in a single result set.

```
carpenter_workshop=# CREATE ROLE workshop_readonly;
CREATE ROLE workshop_order_entry;
CREATE ROLE
CREATE ROLE
carpenter_workshop=#
```

**Screenshot 15: Creating the two roles**

Shows `CREATE ROLE workshop_readonly`; and `CREATE ROLE workshop_order_entry`; both completing successfully.

```
carpenter_workshop=# GRANT SELECT ON customers, orders, order_stages, material_usage
TO workshop_readonly;
GRANT
```

#### Screenshot 16: Granting SELECT to workshop\_readonly

Shows the start of the GRANT SELECT command on all four tables directed to workshop\_readonly.

```
carpenter_workshop=# GRANT SELECT ON customers, orders, order_stages, material_usage
TO workshop_readonly;
GRANT
carpenter_workshop=# GRANT SELECT ON customers, orders, order_stages, material_usage
TO workshop_order_entry;
GRANT INSERT ON orders, order_stages, material_usage
TO workshop_order_entry;
GRANT
GRANT
```

#### Screenshot 17: Granting SELECT and INSERT to workshop\_order\_entry

Shows both GRANT SELECT statements (to workshop\_readonly and workshop\_order\_entry) followed by GRANT INSERT to workshop\_order\_entry, each confirmed with GRANT.

```
carpenter_workshop=# CREATE USER workshop_viewer WITH PASSWORD 'viewer123';
CREATE USER workshop_employee WITH PASSWORD 'employee123';
CREATE ROLE
CREATE ROLE
```

#### Screenshot 18: Creating the login users

Shows CREATE USER workshop\_viewer WITH PASSWORD 'viewer123'; and CREATE USER workshop\_employee WITH PASSWORD 'employee123';.

```
carpenter_workshop=# GRANT workshop_readonly TO workshop_viewer;
GRANT workshop_order_entry TO workshop_employee;
GRANT ROLE
GRANT ROLE
```

#### Screenshot 19: Assigning roles to users

Shows GRANT workshop\_readonly TO workshop\_viewer; and GRANT workshop\_order\_entry TO workshop\_employee;.

```
postgres=# \c carpenter_workshop workshop_viewer
You are now connected to database "carpenter_workshop" as user "workshop_viewer".
```

#### Screenshot 20: Connecting as workshop\_viewer

Shows \c carpenter\_workshop workshop\_viewer, confirming the session is now connected as the read-only user.

```
carpenter_workshop=> SELECT * FROM orders;
 order_id | customer_id | pieces | order_date
-----+-----+-----+-----
          1 |             1 |      19 | 2024-01-08
          2 |             1 |      14 | 2024-01-19
          3 |             2 |      15 | 2024-02-02
(3 rows)
```

#### Screenshot 21: Read-only SELECT succeeds

Shows SELECT \* FROM orders; succeeding for workshop\_viewer, proving the SELECT grant is active.

```
carpenter_workshop=> INSERT INTO orders VALUES (4, 1, 10, '2024-03-10');
ERROR:  permission denied for table orders
carpenter_workshop=>
```

#### Screenshot 22: Read-only INSERT correctly denied

Shows an INSERT attempt by workshop\_viewer failing with ERROR: permission denied for table orders, as expected for a read-only role.



```
carpenter_workshop=> \c carpenter_workshop workshop_employee
You are now connected to database "carpenter_workshop" as user "workshop_employee".
carpenter_workshop=>
```

### Screenshot 23: Connecting as workshop\_employee

Shows \c carpenter\_workshop workshop\_employee, confirming the session is now connected as the order-entry user.

```
carpenter_workshop=> SELECT current_user;
current_user
-----
workshop_employee
(1 row)
```

### Screenshot 24: Confirming the active user

Shows SELECT current\_user; returning workshop\_employee.

```
carpenter_workshop=> INSERT INTO orders VALUES (4, 1, 10, '2024-03-10');
INSERT 0 1
carpenter_workshop=> SELECT * FROM orders;
 order_id | customer_id | pieces | order_date
-----+-----+-----+-----
      1 |           1 |      19 | 2024-01-08
      2 |           1 |      14 | 2024-01-19
      3 |           2 |      16 | 2024-02-02
      4 |           1 |      10 | 2024-03-10
(4 rows)
```

### Screenshot 25: Order-entry INSERT succeeds

Shows INSERT INTO orders VALUES (4, 1, 10, '2024-03-10'); succeeding (INSERT 0 1), followed by SELECT \* FROM orders; showing the new 4th row.

```
carpenter_workshop=> REVOKE INSERT ON orders, order_stages, material_usage
FROM workshop_order_entry;
WARNING: no privileges could be revoked for "orders"
WARNING: no privileges could be revoked for column "tableoid" of relation "orders"
WARNING: no privileges could be revoked for column "cmax" of relation "orders"
WARNING: no privileges could be revoked for column "xmax" of relation "orders"
WARNING: no privileges could be revoked for column "xmin" of relation "orders"
WARNING: no privileges could be revoked for column "ctid" of relation "orders"
WARNING: no privileges could be revoked for column "order_id" of relation "orders"
WARNING: no privileges could be revoked for column "customer_id" of relation "orders"
WARNING: no privileges could be revoked for column "pieces" of relation "orders"
WARNING: no privileges could be revoked for column "order_date" of relation "orders"
WARNING: no privileges could be revoked for "order_stages"
WARNING: no privileges could be revoked for column "tableoid" of relation "order_stages"
WARNING: no privileges could be revoked for column "cmax" of relation "order_stages"
WARNING: no privileges could be revoked for column "xmax" of relation "order_stages"
WARNING: no privileges could be revoked for column "xmin" of relation "order_stages"
WARNING: no privileges could be revoked for column "ctid" of relation "order_stages"
WARNING: no privileges could be revoked for column "stage_id" of relation "order_stages"
WARNING: no privileges could be revoked for column "order_id" of relation "order_stages"
WARNING: no privileges could be revoked for column "stage" of relation "order_stages"
WARNING: no privileges could be revoked for column "stage_date" of relation "order_stages"
WARNING: no privileges could be revoked for column "status" of relation "order_stages"
WARNING: no privileges could be revoked for "material_usage"
WARNING: no privileges could be revoked for column "tableoid" of relation "material_usage"
WARNING: no privileges could be revoked for column "cmax" of relation "material_usage"
WARNING: no privileges could be revoked for column "xmax" of relation "material_usage"
WARNING: no privileges could be revoked for column "xmin" of relation "material_usage"
WARNING: no privileges could be revoked for column "ctid" of relation "material_usage"
WARNING: no privileges could be revoked for column "usage_id" of relation "material_usage"
WARNING: no privileges could be revoked for column "order_id" of relation "material_usage"
WARNING: no privileges could be revoked for column "material" of relation "material_usage"
WARNING: no privileges could be revoked for column "quantity" of relation "material_usage"
WARNING: no privileges could be revoked for column "unit" of relation "material_usage"
REVOKE
```

### Screenshot 26: Attempting REVOKE as the wrong role

Shows REVOKE INSERT ... FROM workshop\_order\_entry; executed while connected as workshop\_employee. PostgreSQL responds with a WARNING for every table and column stating that no privileges could be revoked, because workshop\_employee is not the object owner.

```
carpenter_workshop=> \c carpenter_workshop workshop_employee
You are now connected to database "carpenter_workshop" as user "workshop_employee".
carpenter_workshop=> INSERT INTO orders VALUES (5, 2, 20, '2024-03-15');
INSERT 0 1
```

#### Screenshot 27: INSERT still succeeds after the REVOKE attempt

*Shows a reconnect as workshop\_employee followed by INSERT INTO orders VALUES (5, 2, 20, '2024-03-15'); succeeding (INSERT 0 1) — proving the REVOKE in Screenshot 26 had no real effect.*

## 6. Problem Statement / Case Study Results and Conclusion.

### Findings

- The four-table schema correctly models the workshop's data, and foreign keys keep every order, stage and material record traceable back to a customer.
- workshop\_readonly correctly allows SELECT but blocks INSERT, confirmed by the permission-denied error in Screenshot 22.
- workshop\_order\_entry correctly allows both SELECT and INSERT, confirmed in Screenshot 25.
- REVOKE only takes effect when issued by the object owner or a superuser. When workshop\_employee attempted to revoke its own role's INSERT privilege, PostgreSQL issued warnings and revoked nothing, and a subsequent INSERT still succeeded (Screenshots 26–27).
- This finding directly supports the objective of explaining privilege revocation during employee handover: revocation must be carried out by an authorized administrator account, not left to the departing employee's own session.

### Conclusion

This case study demonstrates a complete pipeline for building a Carpenter Workshop Custom Order Tracking Database using PostgreSQL. It integrates relational schema design, sample data population, and role-based access control using GRANT and REVOKE, backed by a pg\_dump backup. The practical also surfaced a genuine and important PostgreSQL security behaviour — that only an object's owner or a superuser can effectively revoke a privilege — providing a strong, realistic foundation for understanding database security and role management.

## 7. References

- Official PostgreSQL documentation for CREATE TABLE, CREATE ROLE, CREATE USER, GRANT and REVOKE.
- PostgreSQL documentation on privileges and object ownership (GRANT/REVOKE semantics).
- PostgreSQL documentation for pg\_dump and database backup.

- Course material on Database Security and Role Management, Semester III (SQL & NoSQL).
- psql interactive terminal documentation.

## **Appendix: DBMS Viva Preparation**

This appendix contains 45 viva questions with short, memorable answers, followed by quick-reference material for the viva.

### **Q1. What is DBMS?**

Software that lets us create, store, organise and retrieve data in a structured way (e.g. PostgreSQL).

### **Q2. What is PostgreSQL?**

A free, open-source, object-relational DBMS known for standards compliance and reliability.

### **Q3. What is a database?**

An organised collection of related data stored electronically.

### **Q4. What is a table?**

A structured set of rows and columns used to store data of one entity type.

### **Q5. What is a primary key?**

A column (or set of columns) that uniquely identifies every row in a table.

### **Q6. What is a foreign key?**

A column that references the primary key of another table, linking the two tables.

### **Q7. What is referential integrity?**

A rule ensuring a foreign key value always matches an existing primary key value in the parent table.

### **Q8. Why were four tables used?**

To separate customers, orders, stages and materials so each entity is stored once and linked by foreign keys, avoiding duplication.

### **Q9. Explain the customers table.**

Stores customer\_id (PK), name, city and order\_value for every workshop customer.

### **Q10. Explain the orders table.**

Stores order\_id (PK), customer\_id (FK), pieces ordered and order\_date.

### **Q11. Explain order\_stages.**

Stores stage\_id (PK), order\_id (FK), the production stage, its date and status.

### **Q12. Explain material\_usage.**

Stores usage\_id (PK), order\_id (FK), material name, quantity and unit.

### **Q13. What is normalization?**

The process of organising tables to reduce data redundancy and avoid update anomalies.

### **Q14. What is a relationship between tables?**

A logical link, usually created via a foreign key, showing how rows in one table relate to rows in another.

### **Q15. What is a JOIN?**

An SQL operation that combines rows from two or more tables based on a related column.

**Q16. Why use LEFT JOIN?**

To keep all rows from the left table even when there is no matching row in the right table.

**Q17. What is a role in PostgreSQL?**

A database-level object that can own database objects and hold privileges; a user is a role that can log in.

**Q18. What is RBAC?**

Role-Based Access Control — assigning permissions to roles, then assigning users to roles, instead of granting permissions to each user individually.

**Q19. What is GRANT?**

An SQL command that gives a privilege (e.g. SELECT, INSERT) on a database object to a role.

**Q20. What is REVOKE?**

An SQL command that removes a previously granted privilege from a role.

**Q21. What is SELECT?**

An SQL command used to read/query data from a table.

**Q22. What is INSERT?**

An SQL command used to add new rows to a table.

**Q23. What is UPDATE?**

An SQL command used to modify existing rows in a table.

**Q24. What is DELETE?**

An SQL command used to remove existing rows from a table.

**Q25. What is least privilege?**

A security principle stating every account should have only the minimum permissions needed to perform its job.

**Q26. Why create workshop\_readonly?**

To let certain staff view workshop data without being able to change it.

**Q27. Why create workshop\_order\_entry?**

To let staff who record new orders both view data and insert new order-related rows.

**Q28. What is workshop\_viewer?**

A login user assigned the workshop\_readonly role, so it can only SELECT data.

**Q29. What is workshop\_employee?**

A login user assigned the workshop\_order\_entry role, so it can SELECT and INSERT data.

**Q30. Why did SELECT work for workshop\_viewer?**

Because workshop\_readonly was explicitly granted SELECT on all four tables.

**Q31. Why did INSERT fail for workshop\_viewer?**



Because workshop\_readonly was never granted INSERT, so PostgreSQL blocks the operation.

**Q32. What does "permission denied for table orders" mean?**

PostgreSQL is reporting that the current role lacks the privilege needed for the attempted operation on that table.

**Q33. Why did INSERT initially work for workshop\_employee?**

Because workshop\_order\_entry was explicitly granted INSERT on orders, order\_stages and material\_usage.

**Q34. What happens after REVOKE INSERT?**

Normally the privilege is removed and further INSERTs fail; in this practical, the REVOKE was run by workshop\_employee itself, which is not the object owner, so PostgreSQL issued warnings and revoked nothing — a later INSERT still succeeded.

**Q35. Why is privilege revocation important during employee handover?**

So that an employee who changes role or leaves no longer retains access they no longer need, reducing the risk of unauthorised changes.

**Q36. What is a database backup?**

A saved copy of a database's structure and data that can be used to restore it later.

**Q37. What is pg\_dump?**

A PostgreSQL utility that exports a database (schema and/or data) into a backup file, usually a .sql script.

**Q38. Where was the backup saved?**

As carpenter\_workshop.sql in the Downloads folder.

**Q39. What is the difference between a user and a role in PostgreSQL?**

In PostgreSQL every user is a role; a 'user' specifically refers to a role created WITH LOGIN so it can connect to the database.

**Q40. Can a user have multiple roles?**

Yes — a user can be granted membership in more than one role and inherits the combined privileges.

**Q41. What is authentication?**

The process of verifying who a user is, typically via username and password.

**Q42. What is authorization?**

The process of determining what an authenticated user is allowed to do.

**Q43. What is database security?**

The set of measures (authentication, authorization, roles, grants) that protect data from unauthorised access or modification.

**Q44. Why should permissions be tested?**

To confirm that roles behave as designed before real users rely on them, catching misconfigurations early.

**Q45. What is the main security concept demonstrated by this project?**

Role-based access control using GRANT/REVOKE and the principle of least privilege, including the rule that only an object's owner or a superuser can effectively revoke privileges on it.

**10 Most Important Questions to Memorize**

- Q4. What is a table?
- Q5. What is a primary key?
- Q6. What is a foreign key?
- Q18. What is RBAC?
- Q19. What is GRANT?
- Q24. What is DELETE?
- Q30. Why did SELECT work for workshop\_viewer?
- Q31. Why did INSERT fail for workshop\_viewer?
- Q33. Why did INSERT initially work for workshop\_employee?
- Q44. Why should permissions be tested?

**30-Second Project Explanation**

*My project is a Carpenter Workshop Custom Order Tracking Database implemented in PostgreSQL. I created four related tables: customers, orders, order\_stages and material\_usage. I implemented role-based security using workshop\_readonly and workshop\_order\_entry. The read-only user could view data but could not insert records. The order-entry user could both view and insert records. I then attempted to revoke the INSERT privilege and showed that this only succeeds when issued by the object owner or a superuser — attempting it as the grantee itself left the privilege untouched. This demonstrates GRANT, REVOKE, least privilege and the real access-control considerations behind employee handover.*

**Common Viva Mistakes to Avoid**

- Don't confuse a role with a table — a role controls access, a table stores data.
- Don't say GRANT removes permissions; REVOKE removes permissions.
- Don't say SELECT modifies data — SELECT only reads data.
- Don't forget the difference between a primary key (uniquely identifies a row) and a foreign key (references another table's primary key).
- Don't claim workshop\_viewer can INSERT — it can only SELECT.

- Explain that workshop\_employee had INSERT throughout this practical, because the REVOKE attempt did not succeed.
- Explain the permission-denied error correctly: it means the required privilege was never granted to that role.
- Don't say REVOKE always works instantly — only the object owner or a superuser can successfully revoke a privilege.
- Don't claim the database is an AI system; it is a PostgreSQL DBMS project.

### **Final Practical Checklist**

- ☐ Database created
- ☐ Four tables created
- ☐ Primary keys configured
- ☐ Foreign keys configured
- ☐ Sample data inserted
- ☐ Roles created
- ☐ Users created
- ☐ GRANT tested
- ☐ Read-only permission tested
- ☐ INSERT permission tested
- ☐ REVOKE executed
- ☐ Revoked permission verified
- ☐ Screenshots included
- ☐ Database backup created
- ☐ carpenter\_workshop.sql saved